ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Dedrix Harley Outama**
- **Chinese Name: 歐雄傑**
- **Email: dedrixharley7081@gmail.com**
- **Student ID: 113504010**
- **University: National Central University**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. 594 documents

2. Filtering improves domain relevance and reduces irrelevant noise, making discovered patterns easier to interpret. However, filtering too narrowly can also remove useful cross-category relationships.

AI used for this answer: Yes

If yes, how: I used AI to clarify the concept of domain relevance and provide guidance on interpreting the filtering result. The explanation was written independently.

External references (links):

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html

---

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

Using dropna() would remove 40% of the documents, even though their text is still useful. This is a rather draconian approach because it wastes a large amount of data and could lead to biased results if certain categories are affected more than others.

One alternative is to use unsupervised clustering, which groups documents based on similarities in their text without requiring category labels.

AI used for this answer: Yes

If yes, how: I used AI to understand the disadvantages of dropping missing data and to improve the clarity of my explanation.

External references (links):

https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html

https://scikit-learn.org/stable/auto_examples/text/plot_document_clustering.html

---

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. No. Although the sampled dataset follows roughly the same category distribution, the proportions are not exactly identical.

Ex: sci.med has 594 documents in the original dataset, but its sampled count is approximately 260. Random sampling doesn't guarantee identical category percentages.

2. Underrepresentation of minority categories. If one category contains very few documents, random sampling might select too few of them or even exclude that category entirely. This can lead to biased analysis and poor performance when modeling those less common categories.

AI used for this answer: Yes

If yes, how: AI helped me interpret the category distribution charts and understand the risks of random sampling.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. The number of features increased from 35,482 to 35,788 when stop_words=None was used. This added 306 features because common English words were no longer removed.

2. Adding stopwords back generally makes the matrix less sparse because these words appear in many documents, creating more nonzero values.

AI used for this answer: Yes

If yes, how: I used AI to help interpret the feature counts and understand how stopwords affect matrix sparsity.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

https://scikit-learn.org/stable/modules/feature_extraction.html#using-stop-words

---

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	Number of terms kept (keep == True)
0.0	35,482
0.01	5,089
0.05	1,491

1. 1. It decreases significantly, from 35,482 to 1,491 features, because higher thresholds remove more low-variance terms.

2. For word-count features, words that occur very rarely or have almost no variation across documents tend to have low variance. These are removed first as the threshold increases.

3. They generally contain limited information for distinguishing documents. Removing them reduces dimensionality and computational requirements, although it doesn't guarantee that model accuracy will stay the same.

AI used for this answer: Yes

If yes, how: I used AI to interpret the variance threshold results and understand how removing low-variance features affects dimensionality.

External references (links):

https://scikit-learn.org/stable/modules/feature_selection.html?utm_source=chatgpt.com#removing-features-with-low-variance

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.VarianceThreshold.html?utm_source=chatgpt.com

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. My test results show a mixture of both, Pearson ranks several general words highly, such as thanks, looking, and hi, while Spearman identifies more clearly relevant terms like graphics, image, and 3d.

However, neither list contains exclusively graphics-related terminology. Also, negatively correlated words such as people and god should not be interpreted as positive indicators of the graphics category.

2. Pearson measures how closely two variables follow a linear relationship using their original numerical values.

Spearman converts the values into ranks and measures how closely those ranks follow a monotonic relationship (consistently increasing or decreasing, even if not linearly).

3. Take as example, a word whose frequency generally increases with its relevance to a topic, but a few documents contain unusually large counts.

Those extreme counts can distort Pearson correlation. Spearman may still capture the overall ordering because it works with ranks.

AI used for this answer: Yes

If yes, how: AI helped me compare the correlation results and understand the differences between their correlation.

External references (links):

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

When minSup decreased from 9 to 3, discovered patterns increased from 710 to 9,547, while execution time increased from approximately 0.081 to 1.834 seconds. Lower support allows less-common patterns to qualify, expanding the search space.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

Top-K FAE uses k to request a specified number of highest-ranked patterns. MaxFPGrowth instead uses minSup to determine which patterns are frequent enough to qualify, then returns maximal frequent patterns. Its output size is not fixed in advance.

Answer (3) — Maximal vs standard frequent patterns:

A maximal frequent pattern has no larger frequent superset. Reporting only maximal patterns gives a more compact representation because many smaller frequent subsets do not need to be listed separately. One trade-off is that their individual support counts are not preserved in the maximal-pattern output.

AI used for this answer: Yes

If yes, how: I used AI to understand the differences between Top-K FAE and MaxFPGrowth, interpret the experimental results, and improve the clarity of my explanations.

External references (links):

https://pami-vanitha.readthedocs.io/en/latest/frequentPatternBasicFPGrowth.html

https://www.cs.cornell.edu/boom/2001sp/Yiu/Mafia_Paper_HTML.htm

https://www.sciencedirect.com/science/article/abs/pii/S0952197614001717

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. Each document is represented by 35,482 numerical features. A standard scatter plot can display only two or three numerical coordinates per point, therefore we cannot directly display all 35,482 dimensions in an ordinary 2D or 3D scatter plot without transforming or selecting features.

2. We need dimensionality reduction algorithms.

Example: PCA, t-SNE, and UMAP. They transform high-dimensional data into a smaller number of dimensions, commonly two or three, making visualization possible.

AI used for this answer: Yes

If yes, how: I used AI to understand why high-dimensional text data cannot be directly visualized in a standard scatter plot and to identify suitable dimensionality reduction methods.

External references (links):

https://scikit-learn.org/stable/modules/decomposition.html

https://scikit-learn.org/stable/modules/manifold.html

https://umap-learn.readthedocs.io/en/latest

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:
1. My generated plots show that UMAP reveals more distinct group structures, while PCA compresses most documents into overlapping regions.

2. PCA is a linear method that maximizes retained variance. It doesn't specifically try to separate the document categories. t-SNE and UMAP use nonlinear approaches that can reveal neighborhood relationships.

3. t-SNE focuses primarily on preserving local neighborhood relationships. Large distances between clusters in a t-SNE plot don't necessarily mean those document groups are equally far apart in the original feature space.

AI used for this answer: Yes

If yes, how: I used AI to troubleshoot a Unicode encoding error in the notebook and to better understand and compare the PCA, t-SNE, and UMAP visualizations.

External references (links):

https://scikit-learn.org/stable/modules/decomposition.html

https://scikit-learn.org/stable/modules/manifold.html

https://distill.pub/2016/misread-tsne

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. Four columns, 1 per category

2. An algorithm treating these as numerical inputs might incorrectly assume that the categories have a meaningful order and distance.

3. Each category gets its own binary column, so no artificial numerical order is imposed.

AI used for this answer: Yes

If yes, how: I used AI to understand how one-hot binarization works and why integer labels may introduce misleading numerical relationships.

External references (links):

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html

---